1. For this assignment, I used the free version of ChatGPT (GPT-6 Luna) Using a logged-in, but rarely used account as the first LLM. For the second LLM, I used Gemini's "Auto" mode, which has various Gemini 3 versions.
2. For both LLMs, I only sent one request, which consisted of the assignment requirements directly copied from the provided instructions to this assignment, prefaced with "Please follow these instructions using C++:"
3. The code from each LLM will be below
   Start of ChatGPT's code

```cpp
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

class Sudoku {
private:
    static const int SIZE = 9;
    int grid[SIZE][SIZE];

    // Checks whether a number can be placed in a cell.
    bool isValid(int row, int col, int num) const {
        // Check the row.
        for (int c = 0; c < SIZE; c++) {
            if (grid[row][c] == num) {
                return false;
            }
        }

        // Check the column.
        for (int r = 0; r < SIZE; r++) {
            if (grid[r][col] == num) {
                return false;
            }
        }

        // Find the top-left corner of the 3x3 block.
        int blockRow = (row / 3) * 3;
        int blockCol = (col / 3) * 3;
```

```
    // Check the 3x3 block.
    for (int r = blockRow; r < blockRow + 3; r++) {
        for (int c = blockCol; c < blockCol + 3; c++) {
            if (grid[r][c] == num) {
                return false;
            }
        }
    }

    return true;
}

// Finds the next empty cell.
bool findEmptyCell(int &row, int &col) const {
    for (row = 0; row < SIZE; row++) {
        for (col = 0; col < SIZE; col++) {
            if (grid[row][col] == 0) {
                return true;
            }
        }
    }

    return false;
}

// Recursively searches for every solution.
long long solveRecursive() {
    int row, col;

    // If there are no empty cells, a solution has been found.
    if (!findEmptyCell(row, col)) {
        printGrid();
        return 1;
    }

    long long solutions = 0;

    // Try every possible digit from 1 through 9.
    for (int num = 1; num <= 9; num++) {
        if (isValid(row, col, num)) {
```

```
            // Place the candidate number.
            grid[row][col] = num;

            // Recursively solve the remaining puzzle.
            solutions += solveRecursive();

            // Backtrack and try another candidate.
            grid[row][col] = 0;
        }
    }

    return solutions;
}

// Checks whether the original puzzle has conflicting clues.
bool hasValidStartingGrid() {
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            int num = grid[row][col];

            if (num == 0) {
                continue;
            }

            // Temporarily remove the clue to check for conflicts.
            grid[row][col] = 0;

            bool valid = isValid(row, col, num);

            // Restore the original clue.
            grid[row][col] = num;

            if (!valid) {
                return false;
            }
        }
    }

    return true;
}
```

```
public:
    // Initializes an empty Sudoku grid.
    Sudoku() {
        for (int row = 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
                grid[row][col] = 0;
            }
        }
    }

    // Reads a Sudoku puzzle from a file.
    bool loadPuzzle(const string &filename) {
        ifstream inputFile(filename);

        if (!inputFile.is_open()) {
            cout << "Error: Could not open " << filename << endl;
            return false;
        }

        for (int row = 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
                string cell;

                if (!(inputFile >> cell)) {
                    cout << "Error: Invalid puzzle format in "
                         << filename << endl;
                    inputFile.close();
                    return false;
                }

                if (cell == "_") {
                    grid[row][col] = 0;
                } else if (cell.length() == 1 &&
                           cell[0] >= '1' && cell[0] <= '9') {
                    grid[row][col] = cell[0] - '0';
                } else {
                    cout << "Error: Invalid cell in "
                         << filename << endl;
                    inputFile.close();
```

```
                return false;
            }
        }
    }

    // Ensure the file contains exactly 81 cells.
    string extra;

    if (inputFile >> extra) {
        cout << "Error: Too many cells in "
             << filename << endl;
        inputFile.close();
        return false;
    }

    inputFile.close();
    return true;
}

// Prints the Sudoku grid.
void printGrid() const {
    for (int row = 0; row < SIZE; row++) {
        if (row % 3 == 0) {
            cout << "+-------+-------+-------+" << endl;
        }

        for (int col = 0; col < SIZE; col++) {
            if (col % 3 == 0) {
                cout << "| ";
            }

            if (grid[row][col] == 0) {
                cout << "_ ";
            } else {
                cout << grid[row][col] << ' ';
            }
        }

        cout << "|" << endl;
    }
```

```
        cout << "+-------+-------+-------+" << endl;
    }

    // Solves the puzzle and prints all solutions.
    void solve() {
        if (!hasValidStartingGrid()) {
            cout << "No solution found" << endl;
            return;
        }

        long long solutions = solveRecursive();

        if (solutions == 0) {
            cout << "No solution found" << endl;
        } else {
            cout << "Total solutions found: "
                 << solutions << endl;
        }
    }
};

int main() {
    // Process all five puzzle files.
    for (int i = 1; i <= 5; i++) {
        string filename = "puzzle" + to_string(i) + ".txt";

        cout << "\n====================================" << endl;
        cout << "Puzzle file: " << filename << endl;
        cout << "====================================" << endl;

        Sudoku sudoku;

        if (sudoku.loadPuzzle(filename)) {
            cout << "\nOriginal puzzle:" << endl;
            sudoku.printGrid();

            cout << "\nSolution(s):" << endl;
            sudoku.solve();
        }
```

```
        cout << endl;
    }

    return 0;
}
```

End of ChatGPT's code,
Start of Gemini's Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class Sudoku {
private:
    char board[9][9];

    // Check if placing 'num' at board[row][col] is valid
    bool isValid(int row, int col, char num) const {
        for (int i = 0; i < 9; ++i) {
            // Check row and column constraints
            if (board[row][i] == num) return false;
            if (board[i][col] == num) return false;
        }

        // Check 3x3 block constraints
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) {
                if (board[startRow + i][startCol + j] == num) return false;
            }
        }

        return true;
    }

    // Recursive backtracking function to find and print all solutions
    void solveRecursive(int row, int col, int& solutionCount) {
        // Base case: if we've moved past the last row, a solution is found
```

```
        if (row == 9) {
            solutionCount++;
            std::cout << "Solution " << solutionCount << ":\n";
            printBoard();
            std::cout << "\n";
            return;
        }

        // Determine coordinates for the next cell
        int nextRow = (col == 8) ? row + 1 : row;
        int nextCol = (col == 8) ? 0 : col + 1;

        // If the current cell is already filled, move to the next cell
        if (board[row][col] != '_') {
            solveRecursive(nextRow, nextCol, solutionCount);
            return;
        }

        // Try candidates from '1' to '9'
        for (char num = '1'; num <= '9'; ++num) {
            if (isValid(row, col, num)) {
                board[row][col] = num;                          // Place
candidate
                solveRecursive(nextRow, nextCol, solutionCount); // Recurse
                board[row][col] = '_';                          // Backtrack
            }
        }
    }

public:
    Sudoku() {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                board[i][j] = '_';
            }
        }
    }

    // Load puzzle from file (handles whitespace- and underscore-delimited
grid formatting)
```

```
    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                file >> board[i][j];
            }
        }
        return true;
    }

    // Print the 9x9 grid
    void printBoard() const {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                std::cout << board[i][j] << (j == 8 ? "" : " ");
            }
            std::cout << "\n";
        }
    }

    // Public method to initiate solving
    void solve() {
        int solutionCount = 0;
        solveRecursive(0, 0, solutionCount);
        if (solutionCount == 0) {
            std::cout << "No solution found\n";
        }
    }
};

int main() {
    std::string puzzleFiles[] = {"puzzle1.txt", "puzzle2.txt",
"puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};

    for (const auto& filename : puzzleFiles) {
        std::cout << "========================================\n";
```

```
        std::cout << "Puzzle File: " << filename << "\n";
        std::cout << "----------------------------------------\n";

        Sudoku puzzle;
        if (puzzle.loadFromFile(filename)) {
            std::cout << "Initial Puzzle:\n";
            puzzle.printBoard();
            std::cout << "\nSolving...\n\n";
            puzzle.solve();
        } else {
            std::cout << "Error: Could not open file " << filename << "\n";
        }
        std::cout << "\n";
    }

    return 0;
}
```

End of Gemini's code

0. Comparing

- Correctness
  - When comparing the two outputs with the example solution (and each other), the results had no obvious discrepancies, and both continued to yield the same results when compared to other sudoku puzzles I found online for further testing.
  - When I examined the code, I saw that ChatGPT utilized Object-Oriented Programming much more than Gemini's code did. Additionally, Gemini formatted everything with no spacing and only listed how many solutions there were by saying that the below solution was the nth solution, while ChatGPT had clear 3x3 squares and ended the code with how many solutions were found.

- Execution Time
  - Both versions use recursive backtracking to search for Sudoku solutions, so their worst-case execution time is similar and can be exponential in the number of empty cells. ChatGPT's version searches for the next empty cell on each recursive call, while Gemini's version advances through the grid in

row-major order. ChatGPT's version also checks the starting grid and validates the input file before solving, which adds a small amount of work.

- Space Usage
    - Both versions use O(1) space for the fixed 9×9 board and O(81) space for the recursion stack in the worst case. Their space usage is therefore very similar. ChatGPT's version uses a long long solution counter, while the second uses an int.

- Maintainability
    - ChatGPT's version is easier to maintain and troubleshoot because it checks for conflicting starting clues and validates that the input contains exactly 81 valid cells. It also organizes the search around finding an empty cell. Gemini's version is more concise, but it does not check for conflicting clues or handle malformed and incomplete puzzle files robustly. Overall, the first version is more reliable to modify and troubleshoot later.

4. I decided to move forward with ChatGPT's code because it handles input more robustly, including malformed files and conflicting starting clues.
5. Changes go here,
    a. Updated the file-reading logic to handle spaces between digits and underscores in the puzzle files.
    b. Added validation to ensure each puzzle contains exactly 81 cells.
    c. Added countCandidates() to count the valid numbers for an empty cell.
    d. Replaced findEmptyCell() with findBestEmptyCell() to select the empty cell with the fewest valid candidates.
    e. Updated solveRecursive() to use the minimum remaining values (MRV) heuristic.
    f. Added early detection of cells with no valid candidates to reduce unnecessary searching.
    g. Preserved recursive depth-first search with backtracking to solve the puzzles.

h. Preserved the ability to find and print all valid solutions.
i. Updated the puzzle file paths to use "../puzzle" so the program can locate the puzzle files in the parent directory.
j. Added validation to detect conflicting numbers in the original puzzle before solving.